

No fue el Prompt

Data Science & benchmarks para dummies



EDA (Exploratory Data Analysis)

Vocabulario básico

Dataset

La tabla con la que trabajamos: cada fila es un caso, cada columna es un dato sobre ese caso.

Variable numérica

Un valor que es un número con magnitud real. Ej: edad, años de experiencia, salario.

Variable categórica

Una etiqueta o categoría, sin orden numérico. Ej: género, profesión, país.

Feature (entrada)

Lo que le mostramos al modelo para que aprenda a predecir.

Label (salida)

Lo que el modelo tiene que adivinar: la respuesta correcta de cada ejemplo.

Train / Test

Separamos los datos: una parte para enseñarle al modelo, otra para evaluar si aprendió.

¿Regresión, clasificación o clustering?



Regresión

Predecir un **número**.

Ej: “¿cuánto va a costar esta casa?”

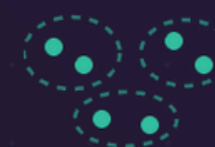


Clasificación

Predecir una **categoría**.

Ej: “¿esta persona es hombre o mujer, según el contexto?”

HOY HACEMOS ESTO



Clustering

Agrupar **sin etiquetas**.

Ej: “¿qué grupos naturales forman estos datos?”

El mundo ya viene sesgado

Antes de entrenar cualquier modelo, miramos los datos tal cual salieron del mundo real — sin tocar nada. **gold_BUG**: 1.717 oraciones reales, validadas a mano.

Pregunta guía: *¿ese desbalance es parejo entre profesiones, o hay algunas que “tiran” mucho más para un lado?*

limpieza: casing + encoding

variable numérica: media/mediana/moda

Outlier: un caso que se aleja mucho del resto de los datos. Con que haya unos pocos alcanza para “tirar” el promedio para su lado — por eso a veces conviene mirar la mediana en vez de la media.

DISTRIBUCIÓN DE GÉNERO EN GOLD_BUG



Y esto es antes de entrenar absolutamente nada. Ahora la pregunta es qué hace un **modelo** con esta realidad.

¿Qué es tokenizar?

Ninguna computadora entiende palabras. Antes de leer una oración, un modelo la parte en piezas chiquitas — los **tokens** — y convierte cada una en un número.

Así el texto se transforma en algo que una red neuronal puede procesar.

ORACIÓN ORIGINAL

The [PERSON] said [PRONOUN] would return tomorrow.

↓ TOKENIZAR

TOKENS

The

[PERSON]

said

[PRONOUN]

would

return

tomorrow

.

↓ CONVERTIR A NÚMEROS

IDS NUMÉRICOS

1996

8034

2056

7241

2052

2709

4826

1012



Modelo

antes que nada

Probar y entender qué pasa - Benchmark de modelos

¿Qué es un embedding?

Un **embedding** es una lista de números (un vector) que representa el *significado* de una palabra o de una oración. Palabras parecidas terminan con vectores parecidos.

Un **Transformer** (como DistilBERT) es la red neuronal que genera esos vectores. Usa “atención”: mira *todas* las palabras del contexto a la vez para decidir qué significa cada una.

Ojo: si el texto con el que se entrenó trae sesgos, el embedding los aprende también — sin que se lo pidamos.

DE TOKENS A SIGNIFICADO



PALABRAS PARECIDAS, VECTORES PARECIDOS... Y A VECES, SESGOS PARECIDOS



attention is all you need

El mecanismo que cambió todo: atención

Antes de 2017, los modelos leían palabra por palabra, en orden, y para cuando llegaban al final de una oración larga ya se habían “olvidado” del principio. El **Transformer** (paper “Attention Is All You Need”, 2017) cambió la estrategia: mira **todas las palabras a la vez** y usa “atención” para decidir qué tan relacionada está cada una con las demás. Es la arquitectura detrás de BERT, GPT, Claude — prácticamente todo lo que hoy llamamos IA generativa.

¿A QUÉ PALABRA LE PRESTA ATENCIÓN “IT”?



Para resolver a qué se refiere “**it**”, el modelo compara esa palabra contra *todas* las demás a la vez y les asigna un peso de atención — acá casi todo el peso cae sobre “**cat**”.

entrada: 768 dim / token

6 capas • 12 cabezas de atención

salida: 768 dim / token

Cada token entra como un vector de 768 números (embedding + posición) y sale como *otro* vector de 768 números, ya “contextualizado”. Nosotros promediamos esos vectores → **1 embedding de 768 dim por oración** (así lo usamos en el resto del proyecto). Y como mira todo a la vez, también entrena en **paralelo** — por eso se pudieron hacer modelos tan grandes.

ESTÚPIDO Y SENSUAL TRANSFORMER



¿Qué son esas 768 dimensiones?

NO SON “768 VECTORES” — ES 1 VECTOR DE 768, POR TOKEN



3 tokens → una matriz de **3 × 768** números (3 vectores, cada uno de 768 — no al revés).

Cada una de esas 768 posiciones es como un “eje” de significado (ninguna con nombre humano). **Más dimensiones = más “lugar”** para guardar distinciones sutiles — por eso los modelos más grandes usan vectores más largos.

Ojo: esto no es lo mismo que el “contexto” (cuántos tokens puede ver a la vez) — son dos parámetros distintos del modelo.

UN LLM GRANDE: MUCHOS “DISTILBERT” APILADOS Y MÁS ANCHOS



Mismo tipo de bloque (capas de atención) repetido 16× más veces, cada uno con vectores 16× más anchos. Modelos más nuevos (GPT-4, Claude) no publican estos números, pero la tendencia general — más capas y más dimensiones — se asocia con más capacidad. No es magia distinta: es la misma arquitectura, mucho más grande.

Encoder, decoder... ¿y DistilBERT qué es?

LO QUE USAMOS HOY



Encoder-only

BERT, DistilBERT

Entiende texto: lo convierte en un vector (embedding). No genera texto nuevo.

Bueno para: clasificar, buscar, comparar significado.



Decoder-only

GPT, Llama

Genera texto, una palabra a la vez, a partir de lo que ya escribió.

Bueno para: chatear, escribir, completar.



Encoder-Decoder

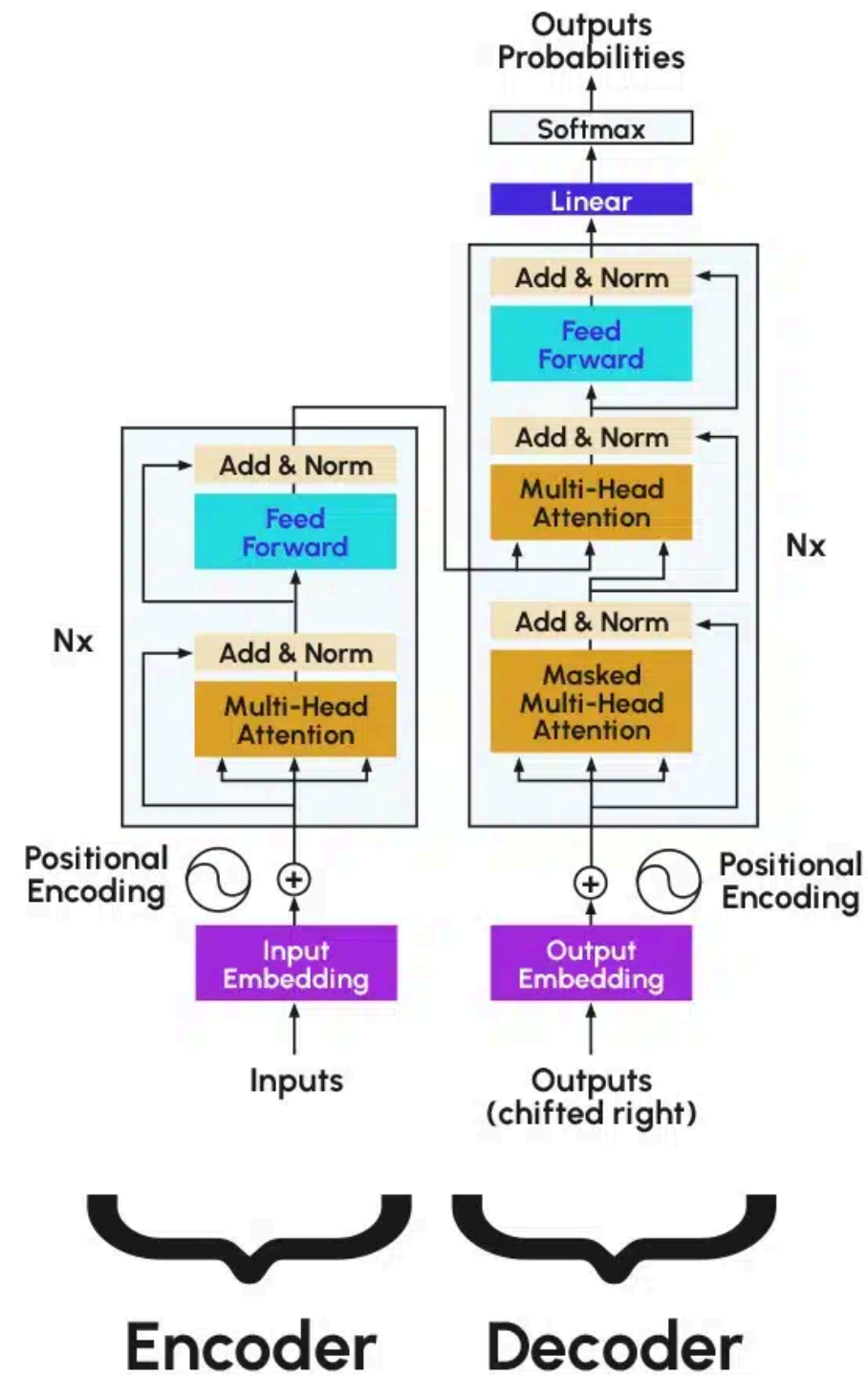
T5, el Transformer original

Entiende con el encoder, y con eso **genera** algo distinto con el decoder.

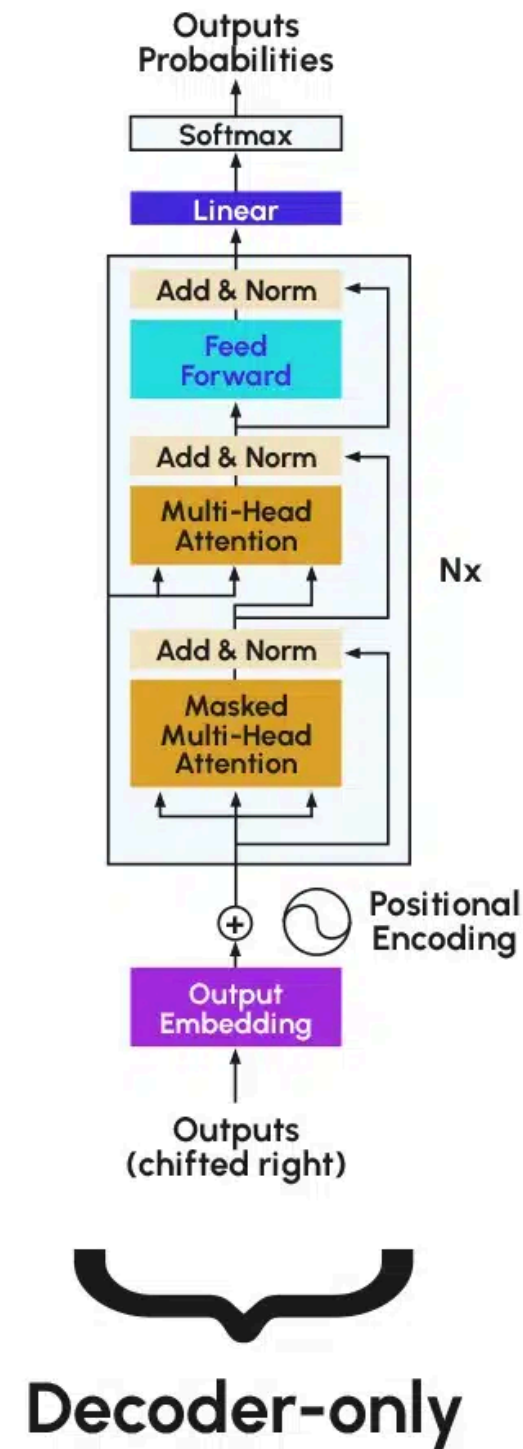
Bueno para: traducir, resumir.

DistilBERT es **encoder-only**: por eso todo lo que hacemos con él es *entender* (embeddings, clasificar) — no le pedimos que *genere* texto nuevo.

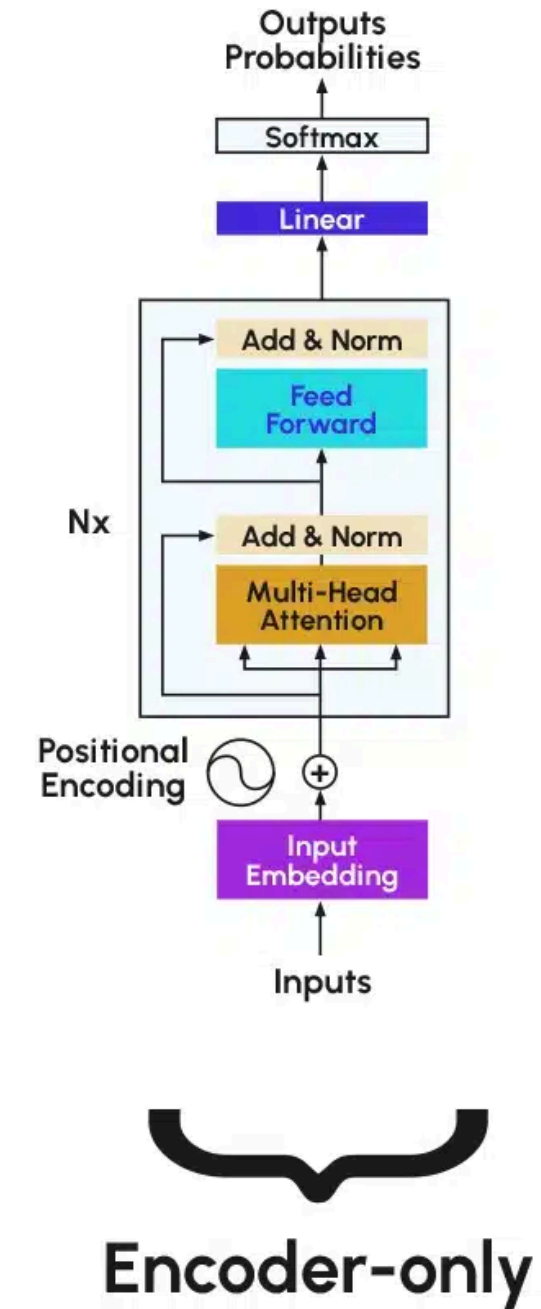
Transformer



GPT*



BERT*

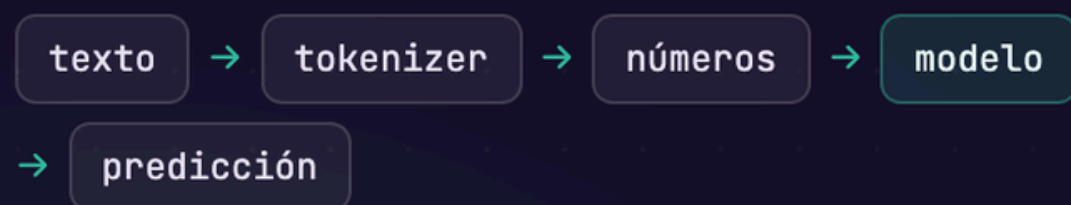


*Illustrative example, exact model architecture may vary slightly

Pre-entrenado vs. sin entrenar

Misma arquitectura (`distilbert-base-uncased`, de Hugging Face) — en un caso con los pesos aprendidos, en el otro con los pesos al azar.

ESTO ES LO QUE “ENTRA” AL MODELO



¿Qué es un peso?

Un número, dentro del “modelo” de arriba, que se ajusta durante el entrenamiento.

66.362.880 pesos

en los dos modelos — la diferencia es el **valor** de cada uno, no la cantidad. Eso es literalmente “entrenar”.

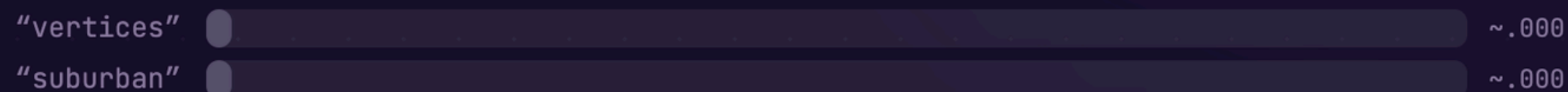
MISMA PREGUNTA, DOS MODELOS

The nurse said **[MASK]** would return tomorrow.

Pre-entrenado (*barras relativas entre sí*)



Sin entrenar



El pre-entrenado ya arma oraciones con sentido — y ese sesgo (she > he para *nurse*, se invierte para *engineer/ceo*) vino incluido, sin que entrenemos nada todavía.

3 benchmarks:

3 formas de comparar helado de vanilla vs. helado elaborado “des-sesgado”

- Ejemplos sueltos (fill-mask, sin entrenar nada): 3 oraciones, le preguntamos directo a cada modelo
- Clasificador entrenado (Logistic Regression): sacamos embeddings y entrenamos nuestro propio clasificador arriba, con gold completo (train/test)
- Fill-mask generalizado, sin entrenar nada: mismo truco que el primero, pero sobre el 87% del dataset (1.496/1.717 oraciones)

**No hay modelo
soberano sin datos
gobernados**



Tres formas de leer el mismo texto



TF-IDF

sin red neuronal

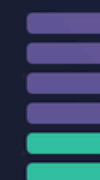
No sabe inglés. Cuenta palabras y aprende qué combinaciones predicen cada clase.



Embeddings congelados

6 capas · 0 entrenables

Ya sabe inglés (pre-entrenado). Lo usamos tal cual, solo para convertir texto en vectores.



Fine-tuning

6 capas · 2 entrenables

Además, dejamos que las últimas capas se ajusten a nuestros datos.

Mismo dataset. Mismo clasificador arriba (Logistic Regression). Lo único que cambia es **cómo representamos el texto**.

Garbage in → bias out



Dataset balanceado

50/50 género · 50/50 estereotipo/anti-estereotipo

↓ mismo tipo de modelo (embeddings + Logistic Regression)

Δ stereotype – chico



Dataset sesgado a propósito

solo ejemplos que siguen el estereotipo (stereotype == 1)

↓ mismo tipo de modelo (embeddings + Logistic Regression)

Δ stereotype – grande

Mismo modelo. Mismo test. Lo único que cambió es **qué le mostramos para entrenar.**

**Vos sos lo que comés
(o entrenás)**

